



GraniteWMS BI Deployment Wizard

Operator Manual

Version 1.6.0

Contents

This table updates automatically in Word – right-click it and choose "Update Field" once, and again after screenshots have been added, since that will shift page numbers.

Contents.....	2
Overview.....	3
Before You Begin.....	3
Getting the Wizard.....	3
Step-by-Step Walkthrough.....	3
Step 1: SQL Server Connection.....	3
Step 2: Live Database Reporting Views.....	4
Step 3: Database Names.....	6
Step 4: Deployment Script Folder.....	7
Step 5: Background Sync Schedule.....	9
Step 6: Deploy and Schedule.....	10
After Deployment Completes.....	11
Troubleshooting Quick Reference.....	11

Overview

The GraniteWMS BI Deployment Wizard sets up and schedules the GraniteWMS BI reporting database on a client's SQL Server instance, without anyone hand-running sqlcmd scripts or configuring Windows Task Scheduler by hand.

This manual walks a Granite implementation consultant through every screen of the wizard, in the order they appear, so a deployment can be completed correctly on the first attempt. It documents the wizard as of version 1.6.0 – check the title bar of the build you're actually running, since panels can gain fields between versions.

Before You Begin

Confirm the following before launching the wizard on the target machine:

- You have obtained the latest versions of the deployment scripts to create views on the production database as well as create the new BI_Database.
- Administrator rights on the machine the wizard runs on – the wizard requests elevation automatically on launch, since registering the scheduled task and running the generated batch file both require it.
- Network access from that machine to the target SQL Server instance hosting GraniteWMS.
- A SQL Server login for the source GraniteWMS database, or a Windows administrator account that can create one – the wizard can create a dedicated login for you if needed.
- The base reporting-views script for the source database (for example Granite_Superset_Views.sql), in case it has not already been run against this environment.
- The folder containing the numbered GraniteWMS BI deployment scripts (01 through 08).
- Which scheduling mechanism you intend to use (Windows Task Scheduler, SQL Server Agent, Granite Scheduler, or none for now) – SQL Server Agent needs Standard or Enterprise edition, so confirm the target instance's edition first if you're planning on that one.

Getting the Wizard

The wizard is distributed as a single, self-contained .exe file – there is nothing to install and no separate runtime to set up on the target machine beforehand.

- Copy the .exe (named with its version, for example GraniteWMS-BI-Deployment-Wizard-v1.6.0.exe) to the machine you will deploy from.
- Right-click the file and choose "Run as administrator".
- The version number is shown in the wizard's title bar at the top of the window – check this if you are reporting an issue.

Step-by-Step Walkthrough

Step 1: SQL Server Connection

The first screen connects to the SQL Server instance and confirms the login can be used to deploy the BI database.

Fields on this screen

- **Server** – the SQL Server instance name (an editable dropdown; it suggests instances it can detect, but any instance name can be typed in directly).
- **SQL Username / SQL Password** – the login to connect with. Once a server is picked, the username field offers a dropdown of that server's existing SQL logins as suggestions when it can see them (checked using the current Windows identity) – it still accepts any name typed directly, including a login that does not exist yet.
- **Test Connection** – opens a connection and checks whether the login has rights to create a new database.

IF THE LOGIN CAN'T CREATE DATABASES

An inline link appears: "I don't have a login with database-creation rights -- create one." Expanding it opens a panel to authenticate once with an admin identity (your own Windows login is usually enough on a freshly installed SQL Server Express instance, or a different SQL admin login), name the new login (defaults to Granite_BI_User), generate or enter its password, and give the source database name so read access is granted there too -- dbcreator alone only covers creating the new BI database, not reading from the existing one via the three-part names (GraniteLive.dbo.vw_BI...) the scripts use. The admin identity is used once, right now, and is never stored; the new login then becomes the credential the rest of the wizard uses, exactly as if you'd typed it in yourself.

GraniteWMS BI Deployment Wizard (v1.5.3)

Step 1 of 6: SQL Server Connection

Step 1 of 6: SQL Server Connection

SQL Server / instance name:

Pick a detected instance, or type one -- e.g. SQL01\GRANITE or 192.168.1.10,1433. The list only includes what's installed on this machine or answers on the network (both best-effort -- a firewalled or Browser-service-off instance won't show up here even though it's perfectly reachable by name).

SQL username:

Password:

Every connection this wizard opens to actually deploy or query the BI data is a fresh, unpooled session built from exactly these credentials (Windows authentication is never used as a fallback here), so nothing here can be silently satisfied by a cached connection instead of the SQL login entered above.

[I don't have a login with database-creation rights -- create one](#)

Step 2: Live Database Reporting Views

GraniteWMS BI reads a set of `dbo.vw_BI_*` reporting views that must already exist in the live GraniteWMS database — this deployment kit does not create them itself, a separate base reporting-views script does. This screen confirms they exist, or deploys them if not.

Fields on this screen

- Source database — the live GraniteWMS database name.
- Check Now — counts the reporting views already present in that database.
- Base reporting-views script — browse to the script file (for example `Granite_Superset_Views.sql`) if the views are missing.
- Use Windows Authentication — tick this if deploying the views needs a different, more privileged account than the login from Step 1.

- **Deploy Views to Source Database** – runs the script and re-checks automatically afterward.

THIS STEP IS REQUIRED

Next is blocked until this check has run and either finds the views already in place or you deploy them successfully. This is deliberate: a missing prerequisite here does not show up until deep into Step 6, after most of the deployment has already run.

GraniteWMS BI Deployment Wizard (v1.5.3)

Step 2 of 6: Live Database Reporting Views

Step 2 of 6: Live Database Reporting Views

This deployment reads \$(SourceDb).dba.vw_BI_ views that must already exist in the live GraniteWMS database -- this BI kit does not create them; a separate base-reporting-views script does. On a fresh or sandbox copy of the live database that never had that script run against it, deployment gets all the way through CREATE DATABASE and then fails with "Invalid object name" for every bi_* table, deep into Panel 6's deploy run -- a much more expensive, confusing failure to hit than catching it here. This check is required: Next is blocked until it runs and either finds the views or you deploy them below.*

Source database (the live GraniteWMS database):

30 vw_BI_* view(s) found in [GraniteLive] -- this prerequisite is already met.

Base reporting-views script (e.g. Granite_Superset_Views.sql):

☐ Use Windows Authentication (the account this wizard is running as) instead of the Step 1 login

Creating views needs more than a read-only grant, so this normally needs an admin login -- tick Windows Authentication if the account running this wizard has that access on this server, otherwise it uses the login from Step 1.

Step 3: Database Names

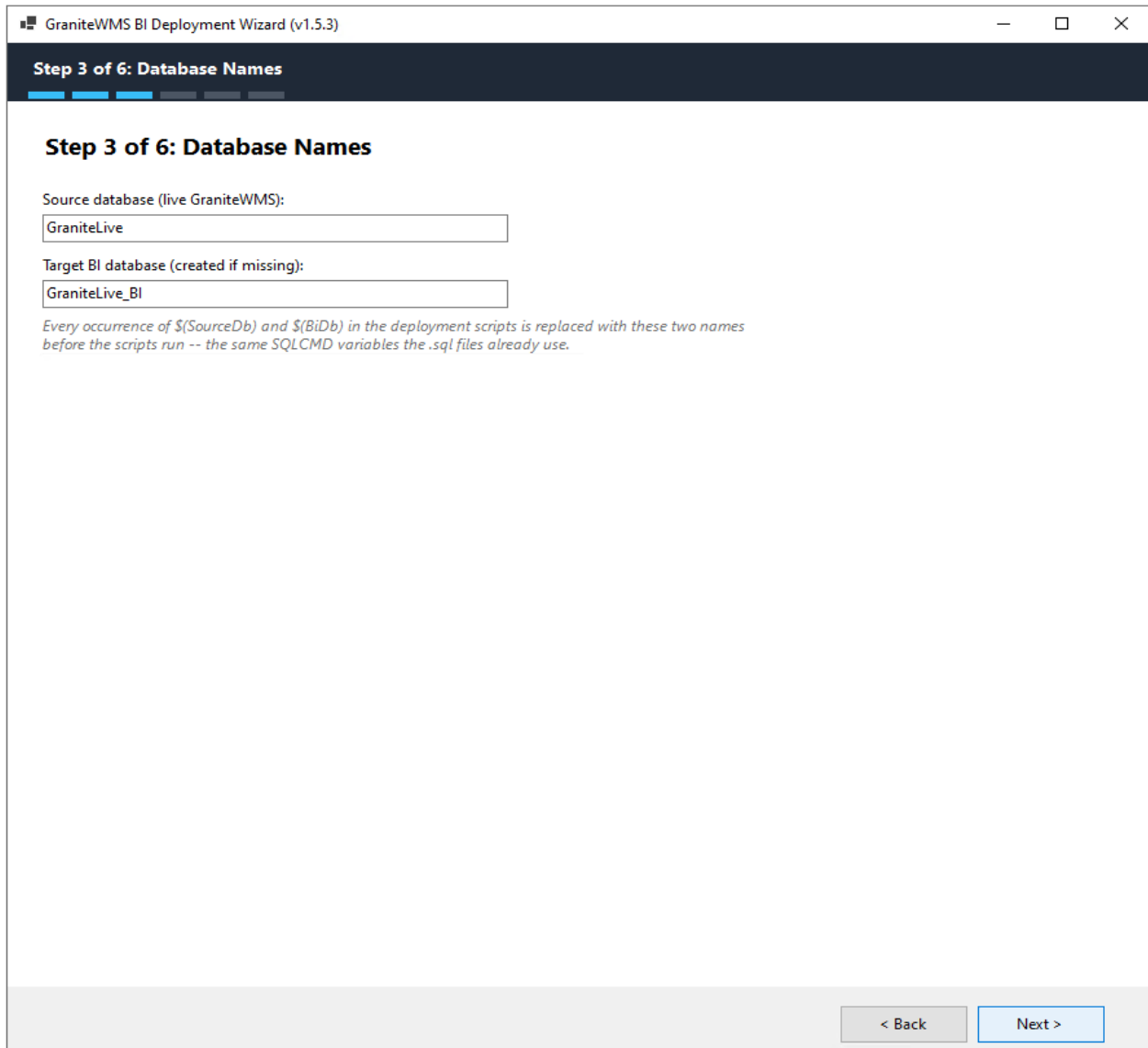
This screen sets the two database names used throughout the rest of the deployment.

Fields on this screen

- Source database – the live GraniteWMS database (defaults to GraniteLive).
- Target BI database – the reporting database the wizard creates and populates (defaults to GraniteLive_BI).

NOTE

If the source database name is changed here after Step 2 already checked a different name, the wizard will ask you to go back and re-check it – a different database could have different reporting-view state.



The screenshot shows a window titled "GraniteWMS BI Deployment Wizard (v1.5.3)". The window has a dark header bar with the text "Step 3 of 6: Database Names". Below the header, the main content area is titled "Step 3 of 6: Database Names". It contains two input fields: "Source database (live GraniteWMS):" with the value "GraniteLive" and "Target BI database (created if missing):" with the value "GraniteLive_BI". Below these fields is a note: "Every occurrence of \$(SourceDb) and \$(BiDb) in the deployment scripts is replaced with these two names before the scripts run -- the same SQLCMD variables the .sql files already use." At the bottom right of the window, there are two buttons: "< Back" and "Next >".

Step 4: Deployment Script Folder

This screen selects the folder holding the numbered GraniteWMS BI deployment scripts. The wizard scans the folder and lists every .sql file it finds, ticked or unticked by default based on what each file is for.

Fields on this screen

- Folder path – browse to the folder containing the 01–08 numbered scripts.
- Script checklist – tick or untick individual files before deploying.

ALWAYS POINT THIS AT A FRESH COPY

Copy the current GraniteWMS BI deployment kit onto the target server first, then browse to that copy here – fresh, for this engagement. Do not point this at a folder that lives with the wizard itself, or a personal sync/Dropbox folder also used for other work: either one can silently drift out of date after a fix is made elsewhere, and the wizard has no way to tell the scripts it is about to run are stale.

WHY SOME FILES START UNTICKED

A script that only sqlcmd itself can run (using :setvar / :r directives) is shown locked and disabled rather than crashing partway through. Files outside the standard deploy order – a one-off collation-fix script, and a standalone SQL Server Agent scheduling script meant for a manual, non-wizard deployment – both start unticked, since neither is part of a normal run, but either can still be ticked on if the situation calls for it. Choosing "SQL Server Agent" on Step 5 registers that scheduled job directly, at whatever interval you pick there – it doesn't need this file ticked on.

GraniteWMS BI Deployment Wizard (v1.5.3)

Step 4 of 6: Deployment Script Folder

Step 4 of 6: Deployment Script Folder

Folder holding the .sql deployment scripts:

C:\Program Files (x86)\GraniteWMS\Granite Rapid Deploy\Granite BI DB\
Browse...

Copy the current GraniteWMS BI deployment kit (the numbered .sql files) onto this server first, then point this step at that copy -- fresh, for this engagement. Do not point this at a folder that lives with the wizard itself, or a personal sync/Dropbox folder you also use for other work: either one can silently drift out of date after a fix is made elsewhere, and this wizard has no way to tell that the scripts it's about to run are stale.

Scripts that will run, in order (untick to skip):

☒ 00_Deploy_All.sql (not supported)
☐ 00_Fix_Collation.sql
☒ 01_Create_BI_Database.sql
☒ 02_Create_BI_Tables.sql
☒ 03_Sync_Engine.sql
☒ 04_Client_Views_rpt.sql
☒ 05_Security_Roles_Logins.sql
☐ 06_Scheduler_SqlAgent.sql
☒ 07_Superset_Compatibility_Views.sql
☒ 08_Indexing.sql

08_Indexing.sql will run.

Files that use SQLCMD-only directives (:setvar, :r) can't be parsed by this wizard's native GO-batch engine and are shown disabled. Files outside the standard deploy order (a master orchestrator, a one-off remediation script, a standalone SQL-Agent scheduler script for a manual, non-wizard deployment) start unticked; tick them back on if you need them. Choosing "SQL Server Agent" on Step 5 registers the Agent job itself, at whatever interval you pick there -- it doesn't need this file ticked on.

< Back
Next >

Step 5: Background Sync Schedule

This screen chooses how the recurring background sync is registered, and sets the interval it runs on. Four mechanisms are available; which fields apply below depends on which one is selected.

Run the recurring sync using:

- Windows Task Scheduler (default) -- works on both SQL Server Express and Standard/Enterprise, registered as a one-time trigger that repeats indefinitely on the interval chosen below.
- SQL Server Agent -- Standard/Enterprise only; there's no Agent service on Express. The wizard checks the connected instance's edition automatically and disables this option,

with an explanation, if it isn't available. Gives job history and alerting inside SSMS that Task Scheduler doesn't.

- Granite Scheduler -- works on both Express and Standard/Enterprise, via Granite's own
-
- ler service and its dbo.ScheduledJobs table. The newest of the three real options -- confirm on your own deployment that the job actually fires (see "After Deployment Completes" below for what to check).
- Skip for now / set up scheduling manually -- nothing is registered automatically. Run_BI_Sync.bat is still generated, ready to wire into whichever mechanism you decide on later.

The wizard checks the connected SQL Server instance's edition as soon as this screen opens, so the SQL Server Agent option is already correctly enabled or disabled before you look at it.

Sync interval:

5, 15, or 30 minutes -- hidden entirely when "Skip for now" is chosen, since no mechanism is being registered for it to apply to.

Windows Task Scheduler -- additional fields (shown only when that option is selected)

- Windows account -- the account the scheduled task runs as (an editable dropdown of this machine's local accounts; type a DOMAIN\user account in directly, since the dropdown only looks up local ones). A bare name like svc-bi is treated as a local account, the same as .\svc-bi.
- Account password -- moving off this step verifies the account and password with Windows automatically (the same check "Verify Account" runs on demand), so a typo is caught here rather than after the whole deployment has already run. This account must itself have a SQL Server login, because the generated Run_BI_Sync.bat calls sqlcmd with -E (trusted/integrated authentication) -- no SQL password is ever written into the .bat file.
- Verify Account -- checks the account/password against Windows on demand.

IF THE ACCOUNT DOESN'T HAVE A SQL LOGIN YET

An inline link appears, the same idea as Step 1's: "This account doesn't have a SQL login yet -- create one." Maps the account to a login (CREATE LOGIN ... FROM WINDOWS) using an admin identity entered once; the wizard grants it EXECUTE on bi.usp_RunSync automatically once the BI database exists, on Step 6. Only needs to be used once per account.

SQL Server Agent -- no additional fields

The job step runs as its owner, the SQL login from Step 1, which already has every right it used to deploy the rest of the BI database. No Windows account is needed.

Granite Scheduler -- no additional fields

Creates a small proxy stored procedure (dbo.usp_RunGraniteBiSync) in the live database that calls into the BI database, and registers it as a row in Granite's own dbo.ScheduledJobs table -- the same kind of footprint Step 2 already leaves with the reporting views. EXECUTE on the BI sync is granted to the "Granite" SQL login the Scheduler service normally connects as, falling back to the public role only if that login isn't present on this instance.

Skip for now -- no additional fields

Run_BI_Sync.bat is still written into the script folder on Step 6, ready to wire into Task Scheduler, SQL Server Agent, or Granite Scheduler manually whenever you decide.

GraniteWMS BI Deployment Wizard (v1.5.3)

Step 5 of 6: Background Sync Schedule

Step 5 of 6: Background Sync Schedule

Run the recurring sync using:

- ☐ Windows Task Scheduler (default) -- works on Express and Standard/Enterprise
- ☐ SQL Server Agent -- Standard/Enterprise only, gives job history and alerting in SSMS
- ☒ Granite Scheduler -- works on Express and Standard/Enterprise, via Granite's own Scheduler service
- ☐ Skip for now / set up scheduling manually -- nothing is registered automatically

This instance is SQL Server Express, which does not include SQL Server Agent -- use Windows Task Scheduler.

Sync interval:

Every 15 minutes

Creates a small proxy procedure (dbo.usp_RunGraniteBiSync) in your live database that calls into the BI database, and registers it in Granite's own dbo.ScheduledJobs table -- the same kind of footprint Step 2 already leaves with the reporting views. EXECUTE on the BI sync is granted to the "Granite" SQL login the Scheduler service normally connects as, falling back to the public role only if that login isn't present on this instance. Newest of the three real options here -- please confirm the job actually fires on a real deployment.

< Back

Next >

Step 6: Deploy and Schedule

The final screen grants the deployment login the rights it needs, then runs the deployment end to end and shows a live log as it happens.

Deployment login rights

Real deployments have turned up missing grants on the Step 1 login more than once -- read access on the source database, then permission to create a stored procedure there for Granite Scheduler. Every "Start Deployment" now begins by granting that login dbcreator (so it can

create the BI database if needed) and db_owner on the source database, using an admin identity entered here – Windows Authentication (the account the wizard itself is running as) or a different SQL admin login, live, never stored. These rights stay standing afterward, not just for this run, so a later redeploy to the same server shouldn't need this step again; granting a login something it already has is harmless.

Fields on this screen

- Start Deployment – grants the rights above, then runs every ticked script in order, then runs an initial sync so the BI tables are populated immediately.
- Cancel – available while a deployment is in progress.
- Log console – shows progress, warnings, and any errors in real time.

What the log actually shows, in order

- Granting the deployment login full rights – the dbcreator/db_owner grant described above.
- Collation check – the BI database's collation is compared to the source database's and repaired automatically if they don't match, including on a redeploy where the BI database already exists; a genuinely first-ever run just notes that the database doesn't exist yet and will be created with the right collation from the start.
- Each ticked script file runs in order, with its own line in the log.
- If the BI database can't be switched into right after being created – some SQL Server instances reassign a freshly created database's ownership away from its creator as a security-hardening policy – the wizard grants access back explicitly, once, and retries automatically before giving up.
- If the BI database already has synced data from an earlier run (checked against bi.SyncLog), a dialog asks whether to re-sync everything now or skip it and let the scheduled sync catch up within about one interval – worth choosing skip on a routine redeploy (adding a view, rerunning after a schema tweak) where a multi-minute resync buys nothing.
- A one-line summary: batches executed, batches failed, files skipped.
- The chosen scheduling mechanism (Step 5) is registered – its own log lines vary by mechanism (writing Run_BI_Sync.bat and a scheduled task, an Agent job, or the Granite Scheduler proxy procedure and job row).

WHAT SUCCESS LOOKS LIKE

Data deployment and scheduler registration are reported as two separate outcomes – a scheduler registration problem after a fully successful data deployment does not mean the deployment failed; the status line says so explicitly ("Deployment complete, but scheduling could not be registered – see log"), and the data already deployed does not need to be redone. Zero failed batches, plus a clean scheduler-registration line, is the expected outcome for a clean run.

GraniteWMS BI Deployment Wizard (v1.5.3)

Step 6 of 6: Deploy and Schedule

Step 6 of 6: Deploy and Schedule

Three separate real deployments each turned up a different missing grant on the Step 1 login -- read access on the source database, then CREATE PROCEDURE there for Granite Scheduler. Every Start Deployment now begins by granting that login dbcreator (so it can create the BI database if it doesn't exist yet) and db_owner on the source (live) database, using the admin identity below -- entered once per run, live, never stored. These rights stay standing afterward, not just for this run, so a later redeploy to this same server shouldn't need this step again. Harmless to re-run: granting a login something it already has does nothing.

☒ Use Windows Authentication (the account this wizard is currently running as)

☐ Use a different SQL admin login

Admin SQL username:

Admin password:

That admin account needs the sysadmin role, or both securityadmin and db_owner of the source database -- either is enough to grant what's needed here.

Start Deployment
Cancel
Deployment complete.

```

[16:59:07] --- 07_Superset_Compatibility_Views.sql ---
[16:59:07] --- 08_Indexing.sql ---
[16:59:07] The BI database already has synced data (last completed sync: 36 minute(s) ago, CompletedWithErrors).
[16:59:08] Skipped the initial sync, as chosen. The scheduled task (every 15 min) will bring the BI tables fully
current on its own.
[16:59:08] Deployment run finished: 215 batch(es) executed, 0 failed, 1 file(s) skipped.
[16:59:08] Registering Granite Scheduler job 'GraniteBiSync' (every 15 min)...
[16:59:08] Granite Scheduler job registered.
  
```

< Back
Close

After Deployment Completes

Once Step 6 finishes successfully, the GraniteWMS BI database exists and is populated from the initial sync (or was already current, if you chose to skip it). What else is in place depends on which scheduling mechanism was chosen on Step 5:

Windows Task Scheduler

- A Run_BI_Sync.bat file has been written into the deployment script folder.
- A Windows Scheduled Task named "GraniteWMS BI Sync - <database name>" is registered, running on the interval chosen on Step 5.

SQL Server Agent

- A SQL Server Agent job is registered on the target instance, running on the interval chosen, owned by the Step 1 SQL login.
- Check its history in SSMS (SQL Server Agent > Jobs) for confirmation it's actually firing.

Granite Scheduler

- A row named after the BI database is registered in the live database's dbo.ScheduledJobs table, with IntervalFormat = MINUTES and the chosen interval as a plain number of minutes.
- Confirm it on the Granite Scheduled Jobs screen (or a direct query against dbo.ScheduledJobs): LastExecutionTime should advance and LastExecutionResult should read SUCCESS after the service's next poll. A freshly registered or corrected row keeps showing whatever result its last real execution attempt produced until the Scheduler service polls again – that's expected, not a new failure; give it up to one interval before re-checking.

Skip for now

- Nothing is registered. Run_BI_Sync.bat has still been written into the script folder, ready to wire into Task Scheduler, SQL Server Agent, or Granite Scheduler by hand whenever you decide.

Whichever mechanism is running, no further action is needed and the wizard doesn't need to stay open or installed on the machine – the BI database keeps itself in sync going forward on its own.

Troubleshooting Quick Reference

If you see...	Do this
"Invalid object name ... vw_BI_..." errors during deployment	The Step 2 reporting-views check was skipped or the views were never deployed. Go back to Step 2 and run Check Now.
A collation error appears during the sync scripts	First confirm Step 4's folder actually holds a fresh, current copy of the deployment kit -- an out-of-date copy of 03_Sync_Engine.sql is the most common real cause. The wizard also repairs a mismatched BI database collation automatically on every run (including redeploy), but that alone won't fix a stale script. From v1.5.8, it also retries a few times if another connection (an open SSMS window is the common case) is briefly holding the BI database open, and reports plainly if it still can't get in.
"The user name or password is incorrect" for the scheduled-task account	Go back to Step 5 and use Verify Account again with the correct password before continuing.
A Windows account error mentioning a name starting with ".\"	The wizard normalizes local account names automatically in most cases. If the error persists, try typing the full computer name instead of ".\" in front of the account name.
"...is not able to access the database ... under the current security context" right after the BI database is created	The wizard already retries this automatically once, using the admin identity entered on Step 6, before it ever reaches you as an error. If it still fails, that admin identity most likely doesn't have enough privilege on this server (needs sysadmin, or securityadmin plus db_owner of the source database) -- try a different admin identity on Step 6.
"The login already has an account under a different user name" during Granite Scheduler or SQL Server Agent registration	This is SQL error 15063 -- the deployment login already owns the BI database under a different mapped name (typically dbo). The wizard resolves this automatically now; if it still surfaces, it usually means the login's mapping on this specific server is unusual enough to be worth a closer look with your DBA.
Granite Scheduler's Scheduled Jobs screen shows LastExecutionResult = "Interval value not valid"	Confirm IntervalFormat is MINUTES with a plain number of minutes as Interval (the wizard writes exactly this) -- CRON and MINUTES are the two real, working formats seen on real installs; SECONDS is not. If the row was only just corrected, LastExecutionResult won't update until the Scheduler service's next poll -- check again after roughly one interval before assuming it's still broken.
A dialog asks whether to skip the initial sync	Appears only when the BI database already has data from an earlier sync. Skip it on a routine redeploy if a scheduler is registered (it'll catch the

	tables up within about one interval); choose to re-sync if no scheduler is registered, or if you need the BI tables current right now.
Deployment finishes with 0 failed batches, but the status line still shows an issue	Scheduler registration is reported separately from data deployment -- a scheduler problem after a clean data deployment does not mean the data needs to be redeployed. Fix whatever the log shows for the registration step and click Start Deployment again; the data deployment step is safe to re-run.
The BI database is left in single-user mode after a deployment run	Fixed from v1.5.7/v1.5.8 on -- the automatic collation check now always releases the database back to multi-user, even if it could not get in on the first try, and reports plainly if something else (an open SSMS window, Superset, another copy of this wizard) is holding the database open. On an older build, or if this still happens, release it manually: ALTER DATABASE [YourBiDbName] SET MULTI_USER;